Benchmarking coding-agent backends on Terminal-Bench/Harbor (20260713)

Overview

We built Phase 3 to answer a practical question: which Claude Code backend routes deliver the best end-to-end coding-agent performance for the money, and where does spend get consumed when attempts fail?

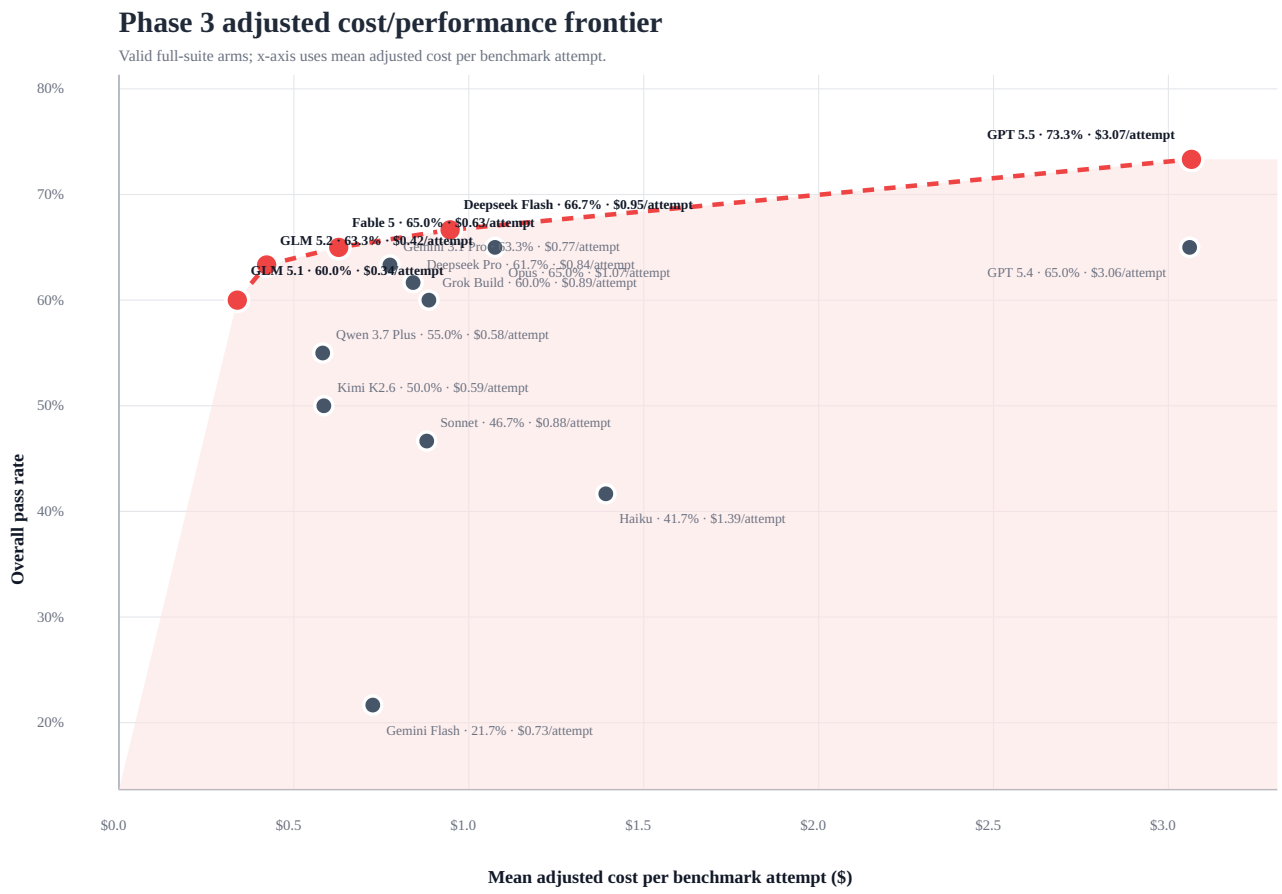
The valid full-suite comparison covers **15 model/backend arms**, **900 benchmark attempts**, and **515 successes** on a fixed Terminal-Bench 2.0 task suite. Claude Code remained the agent harness while the backend route varied across Anthropic, DeepSeek, OpenAI, Google Gemini, GLM/Z.AI, Grok/xAI, Kimi/Moonshot, and Qwen/DashScope.

Like the Databricks coding-agent benchmark, this analysis emphasizes task-level cost/performance rather than headline token price. The important difference is that our Phase 3 accounting layer also quantifies spend and token volume consumed by failed, errored, or operationally unclean attempts.

Main conclusions

1. **No single route dominates the frontier.** The adjusted cost/performance Pareto frontier is **router-glm-5.1, router-glm-5.2, router-anthropic-fable-5, router-deepseek-flash, router-gpt-5.5**.
2. **The highest pass rate came from GPT-5.5**, with **44/60 successes (73.3%)**, but it was one of the most expensive arms at **$183.96** adjusted known cost.
3. **Several lower-cost arms were economically compelling.** GLM 5.1 reached **60.0%** at **$20.30** adjusted cost, GLM 5.2 reached **63.3%** at **$25.32**, and Fable reached **65.0%** at **$37.69**.
4. **Recorded cost materially understated spend.** Recorded cost was **$820.77**, while adjusted known cost was **$972.17**, a known accounting gap of **$151.40** (18.4% over recorded cost).
5. **Failures have a direct economic footprint.** Failure/incomplete spend was **$335.76** (34.5% of adjusted known cost). Broader unclean spend was **$364.29** (37.5%).
6. **Token-volume waste is large enough to operationalize.** Failure/incomplete attempts consumed **213,482,015** input+output tokens; all non-clean outcomes consumed **218,979,987** tokens.

Figure 1: adjusted cost vs. benchmark performance



The chart plots raw pass rate against mean adjusted cost per benchmark attempt. Because each full-suite task has three attempts, the per-attempt number can be multiplied by three for a rough three-attempt-per-task budget.

The frontier shows three practical zones:

- **Low-cost frontier:** router-glm-5.1 and router-glm-5.2 delivered meaningful pass rates at the lowest adjusted costs.
- **Middle frontier:** router-anthropic-fable-5 and router-deepseek-flash offered stronger pass rates without moving into premium-cost territory.
- **Premium frontier:** router-gpt-5.5 delivered the best raw pass rate, but at substantially higher adjusted cost.

Models cluster into capability and efficiency tiers

A pass-rate-only view hides important economic differences. The table below groups arms by performance first, then exposes adjusted cost, clean-success cost, and nonproductive spend.

Arm	Pass rate	Adjusted cost	Cost / clean success	Failure/incomplete spend	Unclean spend	Confidence
router-gpt-5.5	73.3%	$183.96	$4.38	$41.64	$47.93	mixed

Arm	Pass rate	Adjusted cost	Cost / clean success	Failure/incomplete spend	Unclean spend	Confidence
router-deepseek-flash	66.7%	$56.80	$1.42	$13.88	$13.88	medium
router-anthropic-fable-5	65.0%	$37.69	$1.02	$7.33	$17.38	medium
router-anthropic-opus	65.0%	$64.49	$1.65	$30.74	$30.74	mixed
router-gpt-5.4	65.0%	$183.65	$4.83	$34.49	$39.60	low
router-glm-5.2	63.3%	$25.32	$0.72	$6.87	$8.23	medium
router-gemini-3.1-pro	63.3%	$46.47	$1.29	$13.82	$14.46	low
router-deepseek-pro	61.7%	$50.44	$1.36	$12.29	$12.29	medium
router-glm-5.1	60.0%	$20.30	$0.58	$3.94	$4.10	mixed
router-grok-build-0.1	60.0%	$53.15	$1.48	$29.83	$29.83	mixed
router-qwen-3.7-plus	55.0%	$34.94	$1.09	$16.47	$19.16	mixed
router-kimi-k2.6	50.0%	$35.13	$1.17	$22.07	$22.07	low
router-anthropic-sonnet	46.7%	$52.79	$1.89	$23.32	$23.32	mixed
router-anthropic-haiku-sanitized	41.7%	$83.51	$3.34	$54.82	$54.82	high
router-gemini-flash	21.7%	$43.53	$4.84	$24.26	$26.47	mixed

The strongest absolute result was GPT-5.5, but the economical frontier was more diverse. GLM 5.1 and GLM 5.2 were much cheaper, while Fable and DeepSeek Flash occupied a useful middle tier. Opus matched Fable's raw pass rate, but its adjusted cost and failure spend were much higher.

Price-per-token is not the same as price-per-task

A recurring theme in the Databricks report is that developers should not infer end-to-end coding-agent cost from listed token prices alone. Our data shows the same pattern. The benchmark should be priced by completed task attempts, not just model token rates.

Haiku is the clearest example in our run: despite being positioned as a cheaper model class, router-anthropic-haiku-sanitized produced only **41.7%** pass rate while costing **$83.51** adjusted known cost, with **$54.82** spent on failed/incomplete attempts.

DeepSeek Flash shows the opposite nuance: it consumed a large number of tokens on non-clean outcomes (**61.8%** unclean token share), but cache-aware pricing kept the direct unclean spend to **$13.88**.

Failure and unclean spend are first-class benchmark outputs

Traditional benchmark summaries treat failures as quality outcomes. Phase 3 adds a cost lens: failed or operationally unclean attempts also consume money, tokens, and wall-clock time.

- Failure/incomplete spend: **$335.76**.
- Unclean spend, including exception-with-success-signal rows: **$364.29**.
- Failure/incomplete token volume: **213,482,015** input+output tokens.
- Non-clean token volume: **218,979,987** input+output tokens.
- Remaining unresolved cost rows: **29**, all without usable cost or token metadata.

Arm	Unclean spend	Unclean spend share	Unclean tokens	Unclean token share	Pass rate
router-anthropic-haiku-sanitized	$54.82	65.6%	71,926,188	65.7%	41.7%
router-gpt-5.5	$47.93	26.1%	8,501,027	24.9%	73.3%
router-gpt-5.4	$39.60	21.6%	6,595,941	20.1%	65.0%
router-anthropic-opus	$30.74	47.7%	7,630,420	21.8%	65.0%
router-grok-build-0.1	$29.83	56.1%	2,563,589	54.9%	60.0%
router-gemini-flash	$26.47	60.8%	10,092,876	56.7%	21.7%
router-anthropic-sonnet	$23.32	44.2%	17,325,773	40.4%	46.7%
router-kimi-k2.6	$22.07	62.8%	1,644,798	56.2%	50.0%

This view changes how some arms should be interpreted. Kimi, Grok, Qwen, Gemini Flash, and Haiku had comparatively high unclean-spend shares. GPT-5.5 had high absolute unclean spend because its total cost was high, but its share was lower than many mid-tier arms.

Qualitative findings from trajectory and artifact review

The qualitative review helps explain why pure pass-rate and cost summaries are insufficient:

- **Exception-heavy paths can distort both quality and cost.** Sonnet's Phase 3 route produced only **46.7%** pass rate and had many no-token unresolved rows. This should not be read as a general statement that Sonnet is intrinsically weak; it is evidence that the router/harness path had operational issues in this sweep.
- **Reward 1 with exception markers needs its own bucket.** Fable had clean successes plus exception-with-success-signal rows, which is why its failure/incomplete spend is low but broader unclean spend is higher (**$17.38**).
- **Some failed attempts are expensive because the model keeps working.** Opus reached **65.0%**, but failed/incomplete spend was **$30.74**, much higher than Fable at the same raw pass rate.
- **Some failures are cheap in dollars but expensive in tokens.** DeepSeek Flash had **61.8%** unclean token share but only **$13.88** unclean spend.
- **Low pass-rate routes can still burn meaningful budget.** Gemini Flash had **21.7%** pass rate, **$43.53** adjusted cost, and **60.8%** unclean spend share.

The qualitative pattern is that models do not simply fail or succeed. They fail in different operational modes: clean wrong answers, exception failures after real token usage, no-token/no-op signatures, and success signals paired with exception markers. Those modes matter because they have different cost and remediation implications.

What this suggests operationally

The benchmark supports a routing strategy rather than a single default model:

- Use **router-gpt-5.5** when maximum raw success probability matters and premium cost is acceptable.
- Consider **router-anthropic-fable-5**, **router-deepseek-flash**, and **router-glm-5.2** for strong value-oriented routing.
- Consider **router-glm-5.1** where cost per clean success is the main constraint.
- Treat high unclean-spend arms as candidates for harness, timeout, context-management, or route debugging before broad deployment.
- Track cost and token waste as first-class outcomes; otherwise, failed attempts can look cheaper than they really are.

Methodology

- Benchmark: Terminal-Bench 2.0 full suite.
- Scope: 20 tasks × 3 attempts × 15 valid arms = 900 attempts.
- Harness: Claude Code fixed as the agent harness.
- Comparison: valid-only full-suite arms; invalid/quarantined runs excluded.
- Cost metric: adjusted known cost, preserving recorded cost while adding reconstructed missing-cost rows.
- Token-waste metric: input + output tokens by outcome bucket; cache tokens affect cost but are not double-counted in token volume.
- Correctness: verifier/test outcomes, not LLM-as-judge scoring.

Caveats

- Adjusted known cost is benchmark-level accounting, not a substitute for provider invoices.
- Same-arm empirical reconstruction is lower confidence than configured-price reconstruction.

- Some cost rows remain unresolved when neither cost nor token metadata exists.
- Terminal-Bench tasks are not the same as internal production PRs, so this report should guide backend selection experiments rather than claim universal model rankings.